

FLOWSTLC: An Information Flow Control Type System Based On Graded Modality

1st Zhige Chen
Dept. of Computer Science
and Engineering
Southern University of
Science and Technology
Shenzhen, China
12413315@mail.sustech.edu.cn

2nd Junqi Huang
Dept. of Computer Science
and Engineering
Southern University of
Science and Technology
Shenzhen, China
12212226@mail.sustech.edu.cn

3rd Zhiyuan Cao
Dept. of Computer Science
and Engineering
Southern University of
Science and Technology
Shenzhen, China
12311109@mail.sustech.edu.cn

Abstract—In this project, we introduce the design and implementation of a simple functional programming language with a type system for secure information flow. Building on the simply-typed lambda calculus (STLC), we extend the type system with a graded modality over a security semiring. Inspired by *Graded Modal Dependent Type Theory* [1], our system is intended to prevent unauthorized flows of sensitive data into public outputs and to establish noninterference [2] for the core calculus.

Index Terms—Computer security, information flow, noninterference, security-type systems

I. INTRODUCTION

Modern software often handles sensitive data that must remain confidential. The *informational flow control* (or IFC) aims to ensure that high-security inputs of the program do not improperly influence low-security outputs. For example, private user information like passwords should never be leaked to public logs. Formally, the security policy of *noninterference* requires that variations in high-security inputs have no observable effect on low-security outputs. Formally, this requires that variations in high-security inputs result in no observable difference in what an outside observer at low level sees. Violating the IFC principle can lead to information leak, so enforcing noninterference is critical for system security.

Type systems [3] [4] have been proved effective at enforcing noninterference: they associate each value with a *security label* and constrain operations so that a well-typed program is guaranteed not to leak high-security information. For example, consider the following function:

$$\text{foo} = \lambda x : \text{Nat} . \lambda y : \text{Nat} . x \quad (1)$$

Then such coeffect systems might allow the function to be typed as

$$\text{foo} : \text{Nat}^{\text{Sec}} \rightarrow \text{Nat}^{\text{Pub}} \rightarrow \text{Nat}^{\text{Sec}} \quad (2)$$

where the first parameter is marked as a high-security input, and the second parameter is marked as a low-security input. If a low-security result depends on a high-security input, the type system would reject the program. Thus, those security type systems can automatically verify that programs adhere to confidentiality policies.

However, existing IFC type systems can be rather inflexible or coarse-grained. Many systems treat security labels in a rigid, lattice-based way [5], and they can be difficult to combine with other sophisticated type-system constructs. To address these issues, we propose FLOWSTLC, a more flexible and extensible type system that tracks informational flows with *graded modalities*. More specifically, we design a simply typed lambda calculus with integrated *graded necessity* for the semiring $\{\text{Pub} \sqsubseteq \text{Sec}\}$, where Pub represents low-security information and Sec represents high-security information. This system lets us control how values interact using *coeffects* [6] [7], while remaining easy to extend with other program-reasoning constructs such as *usage analysis* [8].

II. FLOWSTLC: THE CORE CALCULUS

A. Syntax

The FLOWSTLC is a graded extension to the simply typed lambda-calculus resembling the Fuzz type system of Reed et al. [9]. It is also similar to coeffect calculi of Brunel et al. [10] and Gaboardi et al. [11].

The syntax of FLOWSTLC is a direct extension of STLC with two additional constructs for introducing and eliminating the graded necessity type $\Box_\ell T$. We also add records, built-in boolean, natural numbers, and unit type to it:

$$\begin{aligned} \text{Term } t ::= & \quad x \mid t \, t \mid \lambda x . t \mid [t] \mid \text{let } [x] = t \text{ in } t \\ & \quad \{l_i = t_i^{i \in 1..n}\} \mid t . l \\ & \quad \text{true} \mid \text{false} \mid \text{if } t \text{ then } t \text{ else } t \mid n \\ & \quad \text{succ } t \mid \text{pred } t \mid \text{iszero } t \mid () \mid \text{let } () = t \text{ in } t \\ \text{Type } T ::= & \quad T \rightarrow^\ell T \mid \Box_r T \\ & \quad \{l_i : T_i^{i \in 1..n}\} \mid \text{Unit} \mid \text{Bool} \mid \text{Nat} \end{aligned}$$

Fig. 1. Syntax of FLOWSTLC

The syntax $[t]$ promotes a term to a graded modality, and $\text{let } [x] = t_1 \text{ in } t_2$ eliminates the modality by checking whether t_2 uses the unboxed value at its grade and then substituting it into t_2 . The function type $T \rightarrow^\ell T$ records how the function

uses its argument. The graded modality $\Box_\ell T$ is parameterized by a security grade ℓ from the semiring $\mathbb{S} = \text{Pub} \sqsubseteq \text{Sec}$. Records, projections, booleans, natural numbers, and unit are standard extensions.

Typing judgements are of the regular form $\Gamma \vdash t : T$ with the typing contexts of the form:

$$\text{Context } \Gamma ::= \emptyset \mid \Gamma, x : [T]_\ell$$

Contexts are either empty $\emptyset$, or can be extended with a *graded assumption* $x : [T]_\ell$. For graded assumptions, their grades ℓ capture their substructural behavior by describing how can they be used in a term, in this case, the grade capture the security level of information. We will use $\text{dom}(\Gamma)$ to denote the variables assigned by context Γ .

B. Typing

FLOWSTLC has all three standard typing rules of STLC:

$$\begin{array}{c} \frac{}{\mathbf{0} \cdot \Gamma, x : [T]_1 \vdash x : T} \text{T-VAR} \\[10pt] \frac{\Gamma, x : [T_1]_\ell \vdash t : T_2}{\Gamma \vdash \lambda x : T_1. t : T_1 \rightarrow^\ell T_2} \text{T-ABS} \\[10pt] \frac{\Gamma_1 \vdash t_1 : T_{11} \rightarrow^\ell T_{12} \quad \Gamma_2 \vdash t_2 : T_{11}}{\Gamma_1 + \ell \cdot \Gamma_2 \vdash t_1 t_2 : T_{12}} \text{T-APP} \end{array}$$

Fig. 2. STLC rules in FLOWSTLC

The exchange and weakening rules that permit context permutation and expansion are implicit here; permutation is recorded by a separate lemma (see [Permutation lemma](#)). The differences from standard STLC are that these rules integrate grades into their premises and conclusions. For example, T-VAR enforces that all unused assumptions have grade Sec , while T-ABS records the usage grade of its bound variable in the arrow type. In T-APP, context concatenation combines graded assumptions by semiring addition (see [Appendix A.3](#)).

The next three rules build the structure of the security label semiring:

$$\begin{array}{c} \frac{\Gamma \vdash t : T}{\ell \cdot \Gamma \vdash [t] : \Box_\ell T} \text{T-PRO} \\[10pt] \frac{\Gamma_1 \vdash t_1 : \Box_\ell T_1 \quad \Gamma_2, x : [T_1]_\ell \vdash t_2 : T_2}{\Gamma_1 + \Gamma_2 \vdash \text{let } [x] = t_1 \text{ in } t_2 : T_2} \text{T-LET} \\[10pt] \frac{\Gamma, x : [T_2]_{\ell_1} \vdash t : T_1 \quad \ell_2 \sqsubseteq \ell_1}{\Gamma, x : [T_2]_{\ell_2} \vdash t : T_1} \text{T-APPROX} \end{array}$$

Fig. 3. Grade Rules in FLOWSTLC

The *promotion* rule T-PRO “promotes” a regular term t to a graded modality by propagating its resource requirement to the context by a *scalar multiplication* (for complete definition of scalar multiplication, see the [Appendix A.3](#)). The T-LET rule provides a way to eliminate graded modality via substitution, where graded value is “unboxed” and substituted into a graded

assumption with matching grades. The context concatenation is also used in the conclusion. Finally, the *approximation* rule T-APPROX permits a derivation under grade ℓ_1 to be used under the lower grade ℓ_2 when $\ell_2 \sqsubseteq \ell_1$. For the security instantiation in this report, this orientation is essential: the reverse direction would make the rejected secret-flow example typable.

The rules for record and projection are based on the book of Pierce and Benjamin [12]. Since FLOWSTLC has a simply-typed base, there are no subtyping rules for record types:

$$\begin{array}{c} \frac{\text{for each } i, \Gamma_i \vdash t_i : T_i}{\sum_{i=1}^n \Gamma_i \vdash \{l_i = t_i^{i \in 1..n}\} : \{l_i : T_i^{i \in 1..n}\}} \text{T-RECORD} \\[10pt] \frac{\Gamma \vdash t : \{l_i : T_i^{i \in 1..n}\}}{\Gamma \vdash t.l_i : T_i} \text{T-PROJ} \end{array}$$

Fig. 4. Record Rules in FLOWSTLC

The remaining rules specify the types of booleans, natural numbers, unit values, and conditionals, etc.

$$\begin{array}{c} \frac{}{\mathbf{0} \cdot \Gamma \vdash \text{true} : \text{Bool}} \text{T-TRUE} \\[10pt] \frac{}{\mathbf{0} \cdot \Gamma \vdash \text{false} : \text{Bool}} \text{T-FALSE} \\[10pt] \frac{\Gamma_1 \vdash t_1 : \text{Bool} \quad \Gamma_2 \vdash t_2 : T \quad \Gamma_3 \vdash t_3 : T \quad \ell \sqsubseteq \mathbf{1}}{\ell \cdot \Gamma_1 + \Gamma_2 + \Gamma_3 \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T} \text{T-COND} \\[10pt] \frac{}{\mathbf{0} \cdot \Gamma \vdash 0 : \text{Nat}} \text{T-ZERO} \\[10pt] \frac{\Gamma \vdash t : \text{Nat}}{\Gamma \vdash \text{succ } t : \text{Nat}} \text{T-SUCC} \\[10pt] \frac{\Gamma \vdash t : \text{Nat}}{\Gamma \vdash \text{pred } t : \text{Nat}} \text{T-PRED} \\[10pt] \frac{\Gamma \vdash t : \text{Nat}}{\Gamma \vdash \text{iszero } t : \text{Bool}} \text{T-ISZERO} \\[10pt] \frac{}{\mathbf{0} \cdot \Gamma \vdash () : \text{Unit}} \text{T-UNIT} \\[10pt] \frac{\Gamma_1 \vdash t_1 : \text{Unit} \quad \Gamma_2 \vdash t_2 : T}{\Gamma_1 + \Gamma_2 \vdash \text{let } () = t_1 \text{ in } t_2 : T} \text{T-UNIT-ELIM} \end{array}$$

Fig. 5. Typing Rules for Built-in Booleans, Integers, etc.

A noteworthy point of these rules is the premise $\ell \sqsubseteq \mathbf{1}$ of rule T-COND. This requirement is added to prevent the *control flow attack* via *implicit flow*, in which the attacker uses some form of equality checking and control flow to leak information. For example, the following typing derivation is invalid in our system:

$$\frac{x : [\text{Bool}]_{\text{Pub}} \vdash x : \text{Bool} \quad \vdash 0 : \text{Nat} \quad \vdash 42 : \text{Nat} \quad \text{Sec} \sqsubseteq \mathbf{1}}{x : [\text{Bool}]_{\text{Sec}} \vdash \text{if } x \text{ then } 0 \text{ else } 42 : \text{Nat}}$$

This is the same solution in Abel and Bernardy [13] and Choudhury et al. [14].

C. Operational Semantics

Once the type-checker shows that a program is well-typed, the AST is interpreted using a standard call-by-value strategy. To support the preservation and progress theorems, we specify the operational semantics of FLOWSTLC in the small-step style. We first specify canonical values:

Value $v ::= \lambda x : T.t \mid [v] \mid \{l_i = v_i^{i \in 1..n}\} \mid n \mid \text{true} \mid \text{false} \mid ()$

Fig. 6. Value of FLOWSTLC

The call-by-value reduction relation $t \rightarrow t'$ is then defined as follows:

$$\begin{array}{c}
\frac{t_1 \rightsquigarrow t'_1}{t_1 t_2 \rightsquigarrow t'_1 t_2} \text{E-APP1} \\
\frac{t_2 \rightsquigarrow t'_2}{v_1 t_2 \rightsquigarrow v_1 t'_2} \text{E-APP2} \\
\frac{}{(\lambda x : T.t) v \rightsquigarrow [x \mapsto v]t} \text{E-APPABS} \\
\frac{t \rightsquigarrow t'}{[t] \rightsquigarrow [t']} \text{E-PRO} \\
\frac{t_1 \rightsquigarrow t'_1}{\text{let } [x] = t_1 \text{ in } t_2 \rightsquigarrow \text{let } [x] = t'_1 \text{ in } t_2} \text{E-LET-EVAL} \\
\frac{}{\text{let } [x] = [v] \text{ in } t \rightsquigarrow [x \mapsto v]t} \text{E-LET-UNBOX}
\end{array}$$

Fig. 7. Core Evaluation Rules of FLOWSTLC

The operational semantics of record and projection are specified in a standard call-by-value style:

$$\begin{array}{c}
\frac{}{\{l_i = v_i^{i \in 1..n}\}.l_j \rightsquigarrow v_j} \text{E-PROJRECORD} \\
\frac{t \rightsquigarrow t'}{t.l \rightsquigarrow t'.l} \text{E-PROJ} \\
\frac{t_j \rightsquigarrow t'_j}{\{l_i = v_i^{i \in 1..j-1}, l_j = t_j, l_k = t_k^{k \in j+1..n}\} \rightsquigarrow \{l_i = v_i^{i \in 1..j-1}, l_j = t'_j, l_k = t'_k^{k \in j+1..n}\}} \text{E-RECORD}
\end{array}$$

Fig. 8. Evaluation Rules for Record and Projection

The operational behavior of primitive values, functions, and conditionals is then specified as follows:

$$\begin{array}{c}
\frac{t_1 \rightsquigarrow t'_1}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightsquigarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t_3} \text{E-IF-EVAL} \\
\frac{}{\text{if true then } t_2 \text{ else } t_3 \rightsquigarrow t_2} \text{E-IF-TRUE} \\
\frac{}{\text{if false then } t_2 \text{ else } t_3 \rightsquigarrow t_3} \text{E-IF-FALSE} \\
\frac{t_1 \rightsquigarrow t'_1}{\text{let } () = t_1 \text{ in } t_2 \rightsquigarrow \text{let } () = t'_1 \text{ in } t_2} \text{E-UNIT-ELIM-EVAL} \\
\frac{}{\text{let } () = () \text{ in } t \rightsquigarrow t} \text{E-UNIT-ELIM} \\
\frac{t \rightsquigarrow t'}{\text{succ } t \rightsquigarrow \text{succ } t'} \text{E-SUCC} \\
\frac{n \text{ is a canonical numeral}}{\text{succ } n \rightsquigarrow n + 1} \text{E-SUCC-NAT} \\
\frac{t \rightsquigarrow t'}{\text{pred } t \rightsquigarrow \text{pred } t'} \text{E-PRED} \\
\frac{t \rightsquigarrow t'}{\text{iszero } t \rightsquigarrow \text{iszero } t'} \text{E-ISZERO} \\
\frac{}{\text{pred } 0 \rightsquigarrow 0} \text{E-PRED-ZERO} \\
\frac{n \text{ is a canonical numeral}}{\text{pred } n + 1 \rightsquigarrow n} \text{E-PRED-POSITIVE} \\
\frac{}{\text{iszero } 0 \rightsquigarrow \text{true}} \text{E-ISZERO-ZERO} \\
\frac{n \text{ is a canonical numeral}}{\text{iszero } n + 1 \rightsquigarrow \text{false}} \text{E-ISZERO-POSITIVE}
\end{array}$$

Fig. 9. Evaluation Rules for Built-in Booleans, Integers, etc.

D. Graded Modality

The central innovation of FLOWSTLC is the use of a graded modality $\Box_\ell T$ to track and control information flow. This modality is parameterized by a security grade ℓ drawn from the semiring $\mathbb{S}$, where the ordering signifies that Sec information is more sensitive than Pub .

Intuitively, a term of type $\Box_\ell T$ is a value that should be used according to some security policies, the grade ℓ attached to the modality serves as a security constraint: it specifies the minimum security level of the context necessary to eliminate the modality and access the underlying value.

The utility of this approach stems from the algebraic structure of the security semiring. The semiring operations (meet for addition, join for multiplication) naturally model the combination of security constraints:

- **Context Concatenation** uses semiring addition, the meet operation ($\sqcap$), to combine graded assumptions for the same variable. For example, using a variable in two different parts of a program, one requiring Sec and the other Pub , results in a combined requirement of Pub ($\text{Sec} \sqcap \text{Pub} = \text{Pub}$).
- **Context Scalar Multiplication** uses semiring multiplication, the join operation ($\sqcup$), to strengthen the security

requirements of a context. This is crucial in the T-PRO rule for ensuring that a promoted term does not leak its contents to a lower grade.

Fundamentally, the graded modality $\Box_\ell T$ serves as a security-aware container. The typing rules enforce the intended discipline that the “capability” ℓ needed to open this container is respected; the noninterference theorem below makes the resulting guarantee precise for the core calculus.

E. A Simple Example

We demonstrate the system’s capability of enforcing noninterference by showing that it rejects the following program:

$$\lambda x : \Box_{\text{Sec}} T. \text{let } [y] = x \text{ in } y : \Box_{\text{Sec}} T \rightarrow^{\text{Pub}} T$$

as it attempts to use a Sec value to compute a Pub value. To type this term, we first need to show that

$$x : [\Box_{\text{Sec}} T]_{\text{Pub}} \vdash \text{let } [y] = x \text{ in } y : \Box_{\text{Sec}} T \rightarrow^{\text{Pub}} T$$

then according to the premises of T-LET rule, we need to show the following

- 1) $x : [\Box_{\text{Sec}} T]_{\text{Pub}} \vdash x : \Box_{\text{Sec}} T$
- 2) $y : [T]_{\text{Sec}} \vdash y : T$

Property (1) is immediate, but there is no rule that allows the derivation of (2). The T-VAR rule is inapplicable in this context because it requires that the used variable must have a Pub grade. Consequently, the term is ill-typed.

III. METATHEORY

A. Type Safety

Lemma 1 (Permutation). *If $\Gamma \vdash t : T$ and Δ is a permutation of Γ , then $\Delta \vdash t : T$ and the derivation depth of the latter is the same as the former.*

The permutation proof is by induction on the typing derivation. Preservation and progress additionally require the modal substitution, inversion, canonical-forms, and reduction-compatibility lemmas; the appendix records proof sketches and explicitly identifies these obligations.

Lemma 2 (Well-typed Modal Substitution). *If $\Gamma_v \vdash v : T_1$ and $\Gamma_b, x : [T_1]_\ell \vdash t_2 : T_2$, then $\Gamma_b + \ell \cdot \Gamma_v \vdash [x \mapsto v]t_2 : T_2$. In the E-LET-UNBOX case, inversion of the canonical-box typing (including any final approximation step) exposes the required scaled context before this substitution is applied. No reduction premise is needed, so the lemma does not presuppose preservation.*

We state type safety in the standard two-part form:

Theorem 1 (Type Preservation). *If $\Gamma \vdash t : T$ and $t \rightsquigarrow t'$, then $\Gamma \vdash t' : T$.*

Theorem 2 (Progress). *If $\emptyset \vdash t : T$, then either t is a value or there exists t' such that $t \rightsquigarrow t'$.*

The preservation proof is an induction on the reduction derivation, using the modal/graded substitution lemma (including context scaling) for E-LET-UNBOX and E-APPABS;

the remaining cases use congruence. The progress proof is an induction on typing, using canonical-forms lemmas for functions, boxes, records, and natural numbers. We state these standard inductions but omit their routine case analysis.

B. Strong Normalization

Theorem 3 (Strong Normalization). *If t is a closed and well-typed FlowSTLC term, then every reduction sequence from t terminates (that is, t is strongly normalizing).*

For this recursion-free calculus, the theorem follows by the standard reducibility argument extended with the record, unit, and modal cases; the full induction is omitted here. The termination-insensitive relations below do not by themselves establish normalization, since their expression clauses intentionally do not require termination.

IV. FUNDAMENTAL THEOREMS AND NONINTERFERENCE

We define the type semantics using a logical-relation model following Rajani et al. [15]. The key difference from their work is that our calculus uses a coefficient-based analysis rather than an effect-based analysis.

We first define a type-indexed unary relation $\llbracket T \rrbracket_\nu$ for canonical values, together with an expression relation $\llbracket T \rrbracket_\varepsilon$ whose terminating values inhabit type T . The relations are formally defined mutually:

$$\begin{aligned} \llbracket T \rrbracket_\varepsilon &= \{t \mid \forall v. (t \rightsquigarrow^* v \wedge v \text{ is a value}) \Rightarrow v \in \llbracket T \rrbracket_\nu\} \\ \llbracket \text{Unit} \rrbracket_\nu &= \{()\} \\ \llbracket \text{Bool} \rrbracket_\nu &= \{\text{true}, \text{false}\} \\ \llbracket \text{Nat} \rrbracket_\nu &= \{n \mid n \in \mathbb{N}\} \\ \llbracket T_1 \rightarrow^\ell T_2 \rrbracket_\nu &= \{\lambda x. t \mid \forall v. v \text{ is a value} \wedge [v] \in \llbracket \Box_\ell T_1 \rrbracket_\varepsilon \implies \\ &\quad [x \mapsto v]t \in \llbracket T_2 \rrbracket_\varepsilon\} \\ \llbracket \Box_\ell T \rrbracket_\nu &= \{[v] \mid v \in \llbracket T \rrbracket_\nu\} \\ \llbracket \{l_i : T_i^{i \in 1..n}\} \rrbracket_\nu &= \{\{l_i = v_i^{i \in 1..n}\} \mid \forall i. v_i \in \llbracket T_i \rrbracket_\nu\} \\ \llbracket \Gamma \rrbracket &= \{\gamma \mid \forall x : [T]_\ell \in \Gamma. [\gamma(x)] \in \llbracket \Box_\ell T \rrbracket_\varepsilon\} \end{aligned}$$

Fig. 10. Unary value, expression, and context relations

For the function type $T_1 \rightarrow^\ell T_2$, the value interpretation contains abstractions whose bodies remain in the result expression relation after substitution of every admissible boxed value. The graded modal value interpretation contains boxes $[v]$ whose underlying value satisfies the relation for T . The expression relation requires every terminating reduction to a canonical value in the value relation; it is therefore termination-insensitive. Finally, $\llbracket \Gamma \rrbracket$ relates substitutions pointwise for every graded assumption in Γ .

We also define a binary relation which captures *indistinguishability* from the perspective of an adversary level $\text{adv} \in \mathbb{S}$. The relation is formally defined by the following figure:

$$\begin{aligned}
\llbracket T \rrbracket_{\mathcal{E}}^{adv} &= \{(t_1, t_2) \mid \forall v_1, v_2. (t_1 \rightsquigarrow^* v_1 \wedge t_2 \rightsquigarrow^* v_2 \wedge v_1, v_2 \text{ are values}) \Rightarrow \llbracket T \rrbracket_{\mathcal{V}}^{adv}\} \\
\llbracket \text{Unit} \rrbracket_{\mathcal{V}}^{adv} &= \{((), ())\} \\
\llbracket \text{Bool} \rrbracket_{\mathcal{V}}^{adv} &= \{(b, b) \mid b \in \{\text{true}, \text{false}\}\} \\
\llbracket \text{Nat} \rrbracket_{\mathcal{V}}^{adv} &= \{(n, n) \mid n \in \mathbb{N}\} \\
\llbracket T_1 \rightarrow^\ell T_2 \rrbracket_{\mathcal{V}}^{adv} &= \{(\lambda x. t_1, \lambda x. t_2) \mid \\
&\quad (\forall v, v'. v, v' \text{ are values} \wedge ([v], [v']) \in \llbracket \Box_\ell T_1 \rrbracket_{\mathcal{E}}^{adv} \\
&\quad \implies ((x \mapsto v) t_1, (x \mapsto v') t_2) \in \llbracket T_2 \rrbracket_{\mathcal{E}}^{adv}) \\
&\quad \wedge (\forall v. v \text{ is a value} \wedge [v] \in \llbracket \Box_\ell T_1 \rrbracket_{\mathcal{E}} \implies \\
&\quad \quad [x \mapsto v] t_1 \in \llbracket T_2 \rrbracket_{\mathcal{E}}) \\
&\quad \wedge (\forall v. v \text{ is a value} \wedge [v] \in \llbracket \Box_\ell T_1 \rrbracket_{\mathcal{E}} \implies \\
&\quad \quad [x \mapsto v] t_2 \in \llbracket T_2 \rrbracket_{\mathcal{E}})\} \\
\llbracket \Box_\ell T \rrbracket_{\mathcal{V}}^{adv} &= \begin{cases} \{([v_1], [v_2]) \mid (v_1, v_2) \in \llbracket T \rrbracket_{\mathcal{V}}^{adv}\} & \ell \leq adv, \\ \{([v_1], [v_2]) \mid v_1 \in \llbracket T \rrbracket_{\mathcal{V}} \wedge v_2 \in \llbracket T \rrbracket_{\mathcal{V}}\} & \neg(\ell \leq adv) \end{cases} \\
\llbracket \{l_i : T_i^{i \in 1..n}\} \rrbracket_{\mathcal{V}}^{adv} &= \{(\{l_i = v_i^{i \in 1..n}\}, \{l_i = v_i^{i \in 1..n}\}) \mid \\
&\quad \forall i. (v_i, v_i') \in \llbracket T_i \rrbracket_{\mathcal{V}}^{adv}\} \\
\llbracket \Gamma \rrbracket_{\mathcal{E}}^{adv} &= \{(\gamma_1, \gamma_2) \mid \forall x : [T]_\ell \in \Gamma. \\
&\quad ([\gamma_1(x)], [\gamma_2(x)]) \in \llbracket \Box_\ell T \rrbracket_{\mathcal{E}}^{adv}\}
\end{aligned}$$

Fig. 11. Binary value, expression, and context relations

The binary relation specified here captures *termination-insensitive noninterference* [16]: it constrains observations when both evaluations terminate, but does not require co-termination. Establishing termination-sensitive noninterference would require an additional co-termination argument and is outside the scope of this work.

For function types, the binary relation quantifies over pairs of related boxed arguments and relates the resulting bodies; the two unary side conditions ensure each abstraction is well typed. For a modality, grades observable by *adv* require related underlying values, while grades not observable by *adv* require only unary validity. In the two-point security order, the non-observable case is exactly $adv = \text{Pub}$ and $\ell = \text{Sec}$. Contexts are related pointwise through this modality interpretation.

A. Fundamental Theorems and Noninterference Property

To formalize the noninterference property for our type system, we need a so-called “fundamental theorem” that connects the syntactic typing with semantic interpretation. We define a unary and a binary version of the theorem for the two types of relations above respectively.

Theorem 4 (Unary Fundamental Theorem). *For all well-typed terms $\Gamma \vdash t : T$, if $\gamma \in \llbracket \Gamma \rrbracket$, then $\gamma t \in \llbracket T \rrbracket_{\mathcal{E}}$, where γt denotes capture-avoiding substitution of the terms in the mapping γ .*

The theorem states that, if t is a term of type T under a context Γ , and γ is a unary substitution in the interpretation of Γ , then γt is in the unary relation at type T . Its proof also uses

grade-erasure of the unary context relation: because the unary interpretation of $\Box_\ell T$ is independent of ℓ , the interpretations of contexts that differ only in grades coincide.

However, since the binary relation involves an external observer, we need additional structure to formally specify it. This needed structure is a subclass of semirings which we call *noninterfering semirings*:

Definition 1 (Noninterfering Semiring). *A noninterfering semiring $\mathcal{R}$ is an ordered semiring (the order and both operations are monotone) with the following additional axioms:*

- *Weak idempotence of multiplication:* $r \cdot r \leq r$
- *Antisymmetry:* $(r \leq s) \wedge (s \leq r) \implies r = s$
- *Multiplicative unit is minimal:* $1 \leq r$
- *Additive unit is maximal:* $r \leq 0$

where r, s are two elements of the semiring $\mathcal{R}$.

Proposition 1 (Properties of Noninterfering Semiring). *For an ordered noninterfering semiring and elements r_1, r_2, s , the following properties hold:*

- *Decreasing addition:* if $r_1 \leq r_2$, then $r_1 + s \leq r_2$.
- *Increasing multiplication:* if $r_1 \leq r_2$, then $r_1 \leq s \cdot r_2$ and $r_1 \leq r_2 \cdot s$.

For the security semiring, 1 is minimal and 0 is maximal by definition; these extrema are used in the variable and weakening cases.

Proposition 2 (Security Semiring is Noninterfering). *The induced semiring $\mathbb{S}$ is noninterfering, with*

- *Weak idempotence of multiplication follows from idempotence of $\sqcup$.*
- *Antisymmetry follows from the ordering.*
- *Minimality of 1 and maximality of 0 follow from the definition of $\mathbb{S}$.*

Now we can formally give the binary version of the fundamental theorem. For a general semiring, the statement assumes the context-scaling, context-combination, and reduction-compatibility lemmas needed in the T-PRO, T-LET, record, and conditional cases; these do not follow from the four axioms above alone:

Theorem 5 (Binary Fundamental Theorem). *For all well-typed terms $\Gamma \vdash t : T$ in a calculus parameterized by a noninterfering semiring $\mathcal{R}$ satisfying these compatibility lemmas, if $(\gamma_1, \gamma_2) \in \llbracket \Gamma \rrbracket^{adv}$, then $(\gamma_1 t, \gamma_2 t) \in \llbracket T \rrbracket_{\mathcal{E}}^{adv}$, where each substitution is capture avoiding.*

With the fundamental theorem established, we can now state the final theorem of noninterference:

Theorem 6 (Termination-insensitive noninterference of FLOWSTLC). *For every judgement $x : [T_1]_r \vdash t : \Box_{adv} \mathbb{B}$ with $adv \leq r$ and $adv \neq r$, and closed values $\emptyset \vdash v_1 : T_1$ and $\emptyset \vdash v_2 : T_1$, the pair $([x \mapsto v_1]t, [x \mapsto v_2]t)$ belongs to $\llbracket \Box_{adv} \mathbb{B} \rrbracket_{\mathcal{E}}^{adv}$. In particular, if both evaluations terminate with $[b_1]$ and $[b_2]$, then $b_1 = b_2$. The conclusion is equality of the observable base values, not full β -equality of the two terms.*

Appendix B gives proof sketches for the fundamental and noninterference theorems and records the compatibility lemmas required for a complete formal proof.

V. BIDIRECTIONAL TYPE CHECKING ALGORITHM

While the previous sections defined the declarative typing rules, practical implementation requires an algorithmic system. In this section we introduce the bidirectional typing for FLOWSTLC. The typing rules are defined by two mutually recursive functions for *checking* and *synthesis*, of the form:

$$\begin{aligned} (\text{checking}) \quad & \Phi \vdash t \Leftarrow T \dashv \Delta \\ (\text{synthesis}) \quad & \Phi \vdash t \Rightarrow T \dashv \Delta \end{aligned} \quad (3)$$

Here, we follow a similar approach to Hodas [17] and Polakow [18]. We split the context into two parts:

- Type context Φ : maps variables to types, this is the input to the algorithm.
- Usage context Δ : records exactly the variables used in t and their computed grades, this is the output of the algorithm.

To support the semiring structure of $\mathbb{S}$, we define the following pointwise operations on the usage context Δ :

Definition 2 (Operations on Usage Contexts).

- *Zero* 0 : maps all variables to 0 .
- *Addition* $\Delta_1 + \Delta_2$: returns Δ where $\Delta(x) = \Delta_1(x) + \Delta_2(x)$.
- *Scaling* $r \cdot \Delta$: returns Δ where $\Delta(x) = r \cdot \Delta_1(x)$.
- *Joining* $\Delta_1 \sqcup \Delta_2$: returns Δ where $\Delta(x) = \Delta_1(x) \sqcup \Delta_2(x)$.

Now we introduce the important rules here. For a complete list of the bidirectional typing rules, please refer to appendix.

The variable rule generates a usage of 1 for the variable used and 0 for everything else:

$$\frac{\Phi(x) = T}{\Phi \vdash x \Rightarrow T \dashv \{x : 1\}} \text{BD-VAR}$$

The system transitions from synthesis to checking:

$$\frac{\Phi \vdash t \Rightarrow T_1 \dashv \Delta \quad T_1 = T_2}{\Phi \vdash t \Leftarrow T_2 \dashv \Delta} \text{BD-SWITCH}$$

For function abstractions, we check the function body against its return type. The algorithm verifies that the declared grade r is no greater than the usage grade inferred for the bound variable x :

$$\frac{\Phi, x : T_1 \vdash t \Leftarrow T_2 \dashv \Delta \quad r \leq \Delta(x)}{\Phi \vdash \lambda x : T_1. t \Leftarrow T_1 \dashv \Delta \setminus \{x\}} \text{BD-ABS}$$

The function application synthesizes the function type, checks type of the argument, then scales the argument's usage by the function's input grade r :

$$\frac{\Phi \vdash t_1 \Rightarrow T_1 \dashv \Delta_1 \quad \Phi \vdash t_2 \Leftarrow T_1 \dashv \Delta_2}{\Phi \vdash t_1 t_2 \Rightarrow T_2 \dashv \Delta_1 + (r \cdot \Delta_2)} \text{BD-APP}$$

For the graded modalities, the promotion rule scales the entire usage context of the underlying term by r :

$$\frac{\Phi \vdash t \Leftarrow T \dashv \Delta}{\Phi \vdash [t] \Leftarrow \Box_r T \dashv r \cdot \Delta} \text{BD-PRO}$$

Modality elimination synthesizes the modality type $\Box_r T$ and then checks the body. The rule ensures that the usage of the unboxed variable inside the body fits within the grade r provided by the modality type:

$$\frac{\Phi \vdash t_1 \Rightarrow \Box_r T \dashv \Delta_1 \quad \Phi, x : T \vdash t_2 \Leftarrow T_2 \dashv \Delta_2 \quad r \leq \Delta_2(x)}{\Phi \vdash \text{let } [x] = t_1 \text{ in } t_2 \Leftarrow T_2 \dashv \Delta_1 + (\Delta_2 \setminus \{x\})}$$

For the conditional, we check the condition and both branches. Their usage contexts are combined by semiring addition, conservatively accounting for the requirements of either execution path. This algorithmic rule is schematic: a sound implementation must also enforce the declarative side condition that the condition's context-scaling grade is at most 1 .

$$\frac{\Phi \vdash t \Leftarrow \text{Bool} \dashv \Delta_1 \quad \Phi \vdash t_1 \Leftarrow T \dashv \Delta_2 \quad \Phi \vdash t_2 \Leftarrow T \dashv \Delta_3}{\Phi \vdash \text{if } t \text{ then } t_1 \text{ else } t_2 \Leftarrow T \dashv \Delta_1 + \Delta_2 + \Delta_3}$$

The bidirectional rules for records, natural numbers, booleans, and unit values are omitted for brevity due to their standard nature.

A. A Bidirectional Typing Example

Consider the term $f z$ with the following typing context Φ :

- $f : T \rightarrow^{\text{Sec}} T$, and
- $z : T$

The goal is to synthesize the type and usage context for the application:

$$\Phi \vdash f z \Rightarrow T \dashv \Delta_{\text{total}} \quad (4)$$

Since it is an application, we use the BD-APP rule first. The rule requires us to synthesize the type for the function f and check the argument z against the function's input type. The former comes from a direct application of the rule BD-VAR:

$$\Phi \vdash f \Rightarrow T \rightarrow^{\text{Sec}} T \dashv \{f : \text{Pub}, z : \text{Sec}\} \quad (5)$$

The latter can be achieved by combining the two rules BD-SWITCH and BD-VAR:

$$\Phi \vdash z \Leftarrow T \dashv \{f : \text{Sec}, z : \text{Pub}\} \quad (6)$$

Then we combine the two usage contexts using the formula $\Delta_1 + r \cdot \Delta_2$, where the function's input grade is $r = \text{Sec}$:

$$\begin{aligned} \Delta_{\text{total}} &= \{f : \text{Pub}, z : \text{Sec}\} + \text{Sec} \cdot \{f : \text{Pub}, z : \text{Sec}\} \\ &= \{f : \text{Pub}, z : \text{Sec}\} + \{f : \text{Sec} \cdot \text{Pub}, z : \text{Sec} \cdot \text{Sec}\} \\ &= \{f : \text{Pub}, z : \text{Sec}\} + \{f : \text{Sec}, z : \text{Sec}\} \\ &= \{f : \text{Pub} + \text{Sec}, z : \text{Sec} + \text{Sec}\} \\ &= \{f : \text{Pub}, z : \text{Sec}\} \end{aligned} \quad (7)$$

So the final result is

$$\Phi \vdash f z \Rightarrow T \dashv \{f : \text{Pub}, z : \text{Sec}\} \quad (8)$$

and the typing succeeds.

VI. IMPLEMENTATION

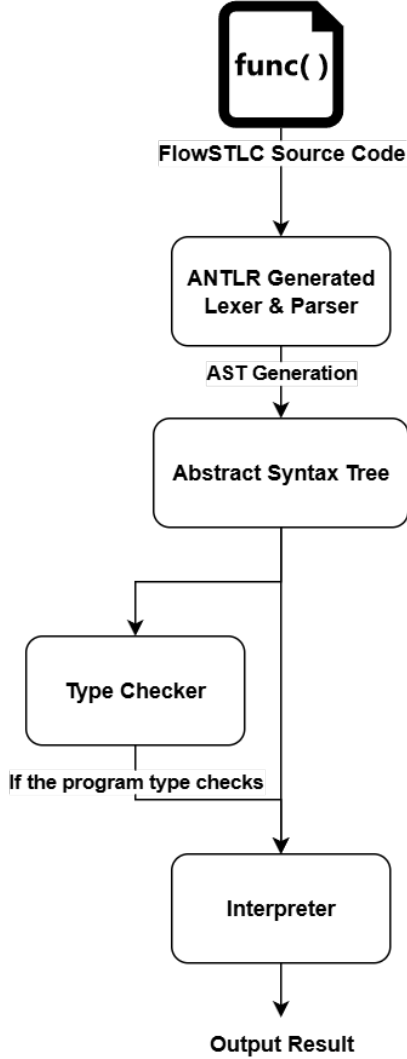


Fig. 12. FLOWSTLC Implementation Pipeline

We implemented FLOWSTLC as a small interpreter pipeline in Java 17, with syntax defined using ANTLR4 [19] and semantics enforced by a typechecker based on the bidirectional typing system developed in the previous section. The front-end is specified by `FlowSTLCParser.g4` and `FlowSTLCParser.g4`, which define a Haskell-like expression language (literals, variables, `let/if`, sequencing, records, arithmetic/boolean operators) plus explicit security annotations (`Sec/Pub`) and modalities. Types include built-in base types (booleans, integers, strings, and unit), record types, modality types, and graded function types written $T_1^r \rightarrow T_2$. The parse tree produced by ANTLR is converted into a small, explicit AST using `ASTBuilder`, which maps each grammar alternative to a dedicated node (e.g., `LetExpr`, `IfExpr`, `FunctionCallExpr`, `ModalityExpr`, `RecordExpr`) and similarly builds the type nodes (`FunctionType`, `ModalityType`, `RecordType`, etc.). This separation lets

later phases operate on stable, type-theory-aligned constructs rather than parser artifacts.

The core static semantics is implemented in `BidirectionalTypeChecker` using two mutually recursive judgments: a synthesis mode and a checking mode. Besides the rules specified in appendix, we also added a few rules to make the language more practical. For example, type checking rules for arithmetic/logical/comparison/(in)equality operators, intrinsic function calls, and sequencing constructs. Type errors are surfaced as `TypeError` exceptions with human-readable diagnostics.

The dynamic semantics is realized by a straightforward environment-based interpreter (`EvalVisitor + Interpreter`) over the AST. Top-level constants evaluate to values and functions evaluate to closures capturing their lexical environment; calls extend the closure environment with evaluated arguments, giving standard call-by-value, lexically scoped execution. Importantly, security labels are enforced statically and erased dynamically: `ModalityExpr` evaluates as a runtime no-op (it simply evaluates its inner expression), matching the intended “filtered label” interpretation where confidentiality is a static property of programs rather than a runtime tag. Ininsics implement basic I/O at the value level. Finally, we provide a language server (LSP4J) to support interactive development: it reparses documents and reruns the typechecker on edits to publish diagnostics, and implements hover and semantic tokens. Build and packaging are automated via Maven, producing shaded runnable jars for both the CLI compiler driver and the language server entry point.

The project implementation can be found at the GitHub repository [nofel248/cs315-term-project](https://github.com/nofel248/cs315-term-project).

VII. EXAMPLES

This section illustrates the practical application of FLOWSTLC through concrete examples of information flow checking.

The first example, adapted from the work on Granule [8], demonstrates the handling of high-security data types.

```

val secret : Int [Sec] = [42]

fun hash : Int [Sec]^Pub -> Int [Sec]
fun hash n = let [x] = n in [x * x * x]

fun main : Unit^Pub -> Int [Sec]
fun main = hash secret
  
```

In this listing, the variable `secret` is defined as a modality with a secret grade. Consequently, the type system restricts operations on this variable to secret-guarded modalities, such as the `hash` function.

Attempts to modify the signature of the `hash` function to downgrade the security level results in type errors. For instance, changing the output grade to `Pub` or removing the modality entirely elicits the following diagnostic:

```

// either
fun hash : Int [Sec]^Pub -> Int [Pub]
fun hash n = let [x] = n in [x * x * x]
  
```

```
// or
fun hash : Int [Sec]^Pub -> Int
fun hash n = let [x] = n in x * x * x
```

Type checking failed:
In **let**-binding: variable 'x' used at **Pub**
which is not > declared modality grade **Sec**

The type checker rejects these programs because they facilitate the flow of secret values into the public domain, which violates the noninterference property.

The second example demonstrates the prevention of control-flow attacks (implicit flows). Consider the following function definitions:

```
fun not_leak : Bool^Pub -> Bool
fun not_leak a = if a then false else true

fun cfa : Bool^Sec -> Bool
fun cfa a = if a then false else true
```

The function `not_leak` is well-typed because the conditional expression depends on a public input. Conversely, the function `cfa` (Control-Flow Attack) attempts to utilize a secret boolean value within the condition. The type checker detects this violation and rejects the code:

Type checking failed:
In `cfa`: parameter 'a' used at **Pub**
which is not > declared input grade **Sec**

To resolve this typing error, the result must be encapsulated within a modality:

```
fun not_longer_cfa : Bool^Sec -> Bool [Sec]
fun not_longer_cfa a = [(if a then false else true)]
```

Since a secret-graded modality cannot be eliminated in a public context, this revised program conforms to the noninterference property and is accepted by the type system.

VIII. RELATED WORK

Language-based IFC has a rich literature. Sabelfeld and Myers [3] survey a variety of security type systems that enforce noninterference via typing. Seminal work by Volpano et al. [16] introduced a static type system for a simple imperative language, ensuring well-typed programs satisfy noninterference. Extensions include JFlow [20] and JIF [21] for Java, which associate security labels with Java types to track flows, and FlowCaml [22] for OCaml, which similarly adds a security type-checker. Generally, these systems label each variable as high or low and enforce rules so that no operation can use a high value in a low context.

More recently, graded type theories have been proposed to generalize such analyses. Graded type systems annotate types with additional information ("grades") to capture various properties of programs. For example, the Granule language [8] uses graded modalities to track the effects and coefficients of program. In information flow use-cases, types are graded by a security lattice, allowing automatic enforcement of noninterference. Moon et al. introduce the Graded Modal

Dependent Type Theory (GRTT) [1], which extends dependent type system with graded modality and show that the grades can be effective at reasoning of programs. This work demonstrates that graded modality can capture fine-grained flow policies in types. Similarly, Marshall and Orchard [23] demonstrated a graded-modal framework that can enforce confidentiality and even integrity simultaneously.

Other approaches include dynamic IFC (e.g. the LIO monad in Haskell [24]) or hybrid systems, but our focus is purely static. Classical security type systems (JIF, FlowCaml, etc.) enforce noninterference via fixed lattice labels. Our project draws on these ideas while adapting graded modalities to a simple functional language to track information flows more flexibly.

IX. FURTHER WORK AND CONCLUSION

In summary, FLOWSTLC presents a simple lambda calculus with graded necessity for enforcing secure information flow. By instantiating grades with the security semiring $\mathbb{S}$, the system tracks data labels at the type level. The stated noninterference result is conditional on the compatibility lemmas listed above; within those assumptions, well-typed core-calculus programs cannot change an observable base result by varying a high-grade input.

While the results are promising, FLOWSTLC in its current form is an *idealized model* with clear limitations. For one, the core calculus is deliberately small and omits many practical language features. It includes records and base types, but no algebraic data types, recursive types, polymorphism, or dependent typing. As observed in prior work, static IFC models with minimal features have historically been considered "too limited or too restrictive to be used in practice" [20]. Thus, our model inherits this limitation of expressiveness. Second, FLOWSTLC exists only as a theoretical calculus and has not been integrated into a production-ready programming language. We consider that bridging this gap is a non-trivial challenge. For example, a recent survey noted that even in a rich language like Rust, implementing a sound IFC system requires "building an ad-hoc effect tracking system using bleeding-edge features of the Rust compiler". Finally, we have not evaluated performance or conducted real-world case studies. There are no benchmarks on typing or runtime costs, nor have we applied FlowSTLC to any non-trivial programs. In contrast, systems like JFlow and Cocoon [25] have shown that static IFC checking can incur negligible overhead. Without such evaluation, the practical impact of FLOWSTLC remains speculative.

Given these limitations, there are many promising directions for future work. We highlight several concrete directions:

- **Extending the type system with dependent types and polymorphism.** One natural extension is to enrich FLOWSTLC with more expressive typing disciplines. In particular, adding *dependent types* could allow security policies to depend on program values and thus encode more precise confidentiality and declassification properties. Prior work (e.g. DepSec for Idris language [26])

shows that dependent types increase the expressiveness of static IFCs. Similarly, introducing *parametric polymorphism* would permit writing generic secure functions without duplicating code.

- **Combining graded modalities with effect, usage, or resource typing.** Another interesting direction is to study how the security grading interacts with other static analyses. Graded type theories generalize both effect systems (via graded monads) and coeffect/usage systems (via graded comonads). For example, FLOWSTLC might be extended with an effect system that tracks side-effects or I/O on secret data (similar to Koka’s effect typing system [27] [28]). Alternatively, one could explore *coeffect-like* analyses where the context is graded alongside security. Integrating graded security labels with *resource- or capability-aware typing* could yield richer guarantees.
- **Embedding FLOWSTLC in a practical language or compiler.** To bring theory closer to practice, an implementation of FLOWSTLC’s typing discipline in a real programming environment is essential. One approach is to create a domain-specific language (DSL) or library for an existing language (e.g. Haskell or OCaml) that enforces these graded security types. Recent work (e.g. [25]) shows that it is possible to add IFC to Rust without modifying the compiler. Similarly, the Granule project [8] demonstrates that a language with graded, linear, and dependent types can be realized in practice. Following these examples, we could adapt FLOWSTLC’s type checker into an implementation (for instance, a GHC plugin) to test on larger codebases. Such an embedding would enable empirical measurement of type-checking performance and runtime costs. It would also allow exploration of practical issues (language interoperability, tooling integration, etc.) that are vital for any real applications of our outcome.

In conclusion, FLOWSTLC provides a formal foundation for secure information flow via graded modal types, but many challenges remain. Future work will pursue richer type features, deeper integration with program reasoning (effects, resources), and practical implementation efforts. We are optimistic that bridging these gaps will bring secure flow typing closer to real-world programming. By pursuing these directions, we hope to develop a mature framework that combines the rigorous guarantees of our type-theoretic approach with the expressiveness and practicality needed for deployed systems.

ACKNOWLEDGMENT

We would like to express our sincere gratitude to the Theoretical Computer Science StackExchange user taquetgauche, who reviewed our early prototype type systems and provided valuable feedback on how to proceed.

We are also deeply thankful to all members of the Theoretical Computer Science Society of SUSTech for their continued support and for the many insightful ideas they shared with us throughout this project.

REFERENCES

- [1] B. Moon, H. Eades III, and D. Orchard, “Graded modal dependent type theory,” in *European Symposium on Programming*. Springer International Publishing Cham, 2021, pp. 462–490.
- [2] G. Smith, “Principles of secure information flow analysis,” in *Malware Detection*. Springer, 2007, pp. 291–307.
- [3] A. Sabelfeld and A. C. Myers, “Language-based information-flow security,” *IEEE Journal on selected areas in communications*, vol. 21, no. 1, pp. 5–19, 2003.
- [4] M. Abadi, A. Banerjee, N. Heintze, and J. G. Riecke, “A core calculus of dependency,” in *Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 1999, pp. 147–160.
- [5] D. E. Denning, “A lattice model of secure information flow,” *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.
- [6] T. Petricek, D. Orchard, and A. Mycroft, “Coeffects: Unified static analysis of context-dependence,” in *International Colloquium on Automata, Languages, and Programming*. Springer, 2013, pp. 385–397.
- [7] —, “Coeffects: a calculus of context-dependent computation,” *ACM SIGPLAN Notices*, vol. 49, no. 9, pp. 123–135, 2014.
- [8] D. Orchard, V.-B. Liepelt, and H. Eades III, “Quantitative program reasoning with graded modal types,” *Proceedings of the ACM on Programming Languages*, vol. 3, no. ICFP, pp. 1–30, 2019.
- [9] J. Reed, M. Gaboardi, and B. Pierce, “Distance makes the types grow stronger: A calculus for differential privacy (extended version).”
- [10] A. Brunel, M. Gaboardi, D. Mazza, and S. Zdancewic, “A core quantitative coeffect calculus,” in *European Symposium on Programming Languages and Systems*. Springer, 2014, pp. 351–370.
- [11] M. Gaboardi, S.-y. Katsumata, D. Orchard, F. Breuvert, and T. Uustalu, “Combining effects and coeffects via grading,” *ACM SIGPLAN Notices*, vol. 51, no. 9, pp. 476–489, 2016.
- [12] B. C. Pierce, *Types and programming languages*. MIT press, 2002.
- [13] A. Abel and J.-P. Bernardy, “A unified view of modalities in type systems,” *Proceedings of the ACM on Programming Languages*, vol. 4, no. ICFP, pp. 1–28, 2020.
- [14] P. Choudhury, H. Eades III, R. A. Eisenberg, and S. Weirich, “A graded dependent type system with a usage-aware semantics,” *Proceedings of the ACM on Programming Languages*, vol. 5, no. POPL, pp. 1–32, 2021.
- [15] V. Rajani and D. Garg, “Types for information flow control: Labeling granularity and semantic models,” in *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*. IEEE, 2018, pp. 233–246.
- [16] D. Volpano, C. Irvine, and G. Smith, “A sound type system for secure flow analysis,” *Journal of computer security*, vol. 4, no. 2-3, pp. 167–187, 1996.
- [17] J. S. HODAS, *Logic programming in intuitionistic linear logic: theory, design, and implementation*. University of Pennsylvania, 1994.
- [18] J. Polakow, “Embedding a full linear lambda calculus in haskell,” *ACM SIGPLAN Notices*, vol. 50, no. 12, pp. 177–188, 2015.
- [19] T. Parr, “The definitive antlr 4 reference,” 2013.
- [20] A. C. Myers, “Jflow: Practical mostly-static information flow control,” in *Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 1999, pp. 228–241.
- [21] K. Pulicino, “Jif: Language-based information-flow security in java,” *arXiv preprint arXiv:1412.8639*, 2014.
- [22] V. Simonet and I. Rocquencourt, “Flow caml in a nutshell,” in *Proceedings of the first APPSEM-II workshop*, 2003, pp. 152–165.
- [23] D. Marshall and D. Orchard, “Graded modal types for integrity and confidentiality,” *arXiv preprint arXiv:2309.04324*, 2023.
- [24] D. Stefan, A. Russo, J. C. Mitchell, and D. Mazières, “Flexible dynamic information flow control in haskell,” in *Proceedings of the 4th ACM Symposium on Haskell*, 2011, pp. 95–106.
- [25] A. Lamba, M. Taylor, V. Beardsley, J. Bambeck, M. D. Bond, and Z. Lin, “Cocoon: Static information flow control in rust,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 166–193, 2024.
- [26] S. Gregersen, S. E. Thomsen, and A. Askarov, “A dependently typed library for static information-flow control in idris,” in *International Conference on Principles of Security and Trust*. Springer, 2019, pp. 51–75.
- [27] D. Leijen, “Koka: Programming with row polymorphic effect types,” *arXiv preprint arXiv:1406.2061*, 2014.

A. Complete Specification of FLOWSTLC

1) Syntax:

Term	t	$::=$	x $t\ t$ $\lambda x.t$ $[t]$ $\mathbf{let}\ [x] = t_1\ \mathbf{in}\ t_2$ $\{l_i = t_i^{i \in 1..n}\}$ $t.l$ $\mathbf{true}$ $\mathbf{false}$ $\mathbf{if}\ t\ \mathbf{then}\ t\ \mathbf{else}\ t$ n $\mathbf{succ}\ t$ $\mathbf{pred}\ t$ $\mathbf{iszero}\ t$ $()$ $\mathbf{let}\ () = t\ \mathbf{in}\ t$	<i>variable</i> <i>application</i> <i>abstraction</i> <i>packing</i> <i>unpacking</i> <i>record</i> <i>projection</i> <i>true constant</i> <i>false constant</i> <i>conditional</i> <i>natural number</i> <i>successor</i> <i>predecessor</i> <i>zero test</i> <i>unit</i> <i>unit elimination</i>
Number	n	$::=$	$n \in \mathbb{N}$	<i>canonical numeral</i>
Type	T	$::=$	$\mathbb{B}$ $\{l_i : T_i^{i \in 1..n}\}$ $T \rightarrow^\ell T$ $\Box_r T$	<i>base type</i> <i>record type</i> <i>function type</i> <i>graded modality</i>
Base type	$\mathbb{B}$	$::=$	$\mathbf{Unit}$ $\mathbf{Nat}$ $\mathbf{Bool}$	<i>unit type</i> <i>natural number type</i> <i>boolean type</i>
Value	v	$::=$	$\lambda x : T.t$ $[v]$ $\{l_i = v_i^{i \in 1..n}\}$ n $\mathbf{true}$ $\mathbf{false}$ $()$	<i>abstraction value</i> <i>boxed value</i> <i>record value</i> <i>natural number value</i> <i>true value</i> <i>false value</i> <i>unit value</i>
Context	Γ	$::=$	$\emptyset$ $\Gamma, x : [T]_r$	<i>empty context</i> <i>graded assumption</i>
Security	ℓ	$::=$	$\mathbf{Sec}$ $\mathbf{Pub}$	<i>high-security label</i> <i>low-security label</i>

2) Security Level Semiring:

Definition 3 (Security Level Semiring). *The security level semiring $\mathbb{S}$ is a two-point lattice of security levels $\mathbb{S} = \{\mathbf{Pub} \sqsubseteq \mathbf{Sec}\}$ with*

- $\mathbf{0} = \mathbf{Sec}$
- $\mathbf{1} = \mathbf{Pub}$
- *Addition as the meet:* $r + s = r \sqcap s$
- *Multiplication as the join:* $r \cdot s = r \sqcup s$

See [Appendix B](#) for a proof that this algebra is indeed a semiring.

3) Auxiliary Definitions:

Definition 4 (Context Concatenation).

$$\begin{aligned}
& \emptyset + \Gamma = \Gamma \\
& \Gamma + \emptyset = \Gamma \\
& (\Gamma, x : [T]_r) + \Gamma' = (\Gamma + \Gamma'), x : [T]_r \quad (x \notin \text{dom}(\Gamma')) \\
& \Gamma + (\Gamma', x : [T]_s) = (\Gamma + \Gamma'), x : [T]_s \quad (x \notin \text{dom}(\Gamma)) \\
& (\Gamma, x : [T]_r) + (\Gamma', x : [T]_s) = (\Gamma + \Gamma'), x : [T]_{r+s}
\end{aligned}$$

Definition 5 (Context Scalar Multiplication).

$$\begin{aligned}
& r \cdot \emptyset = \emptyset \\
& r \cdot (\Gamma, x : [T]_s) = (r \cdot \Gamma), x : [T]_{(r \cdot s)}
\end{aligned}$$

4) *Typing Rules:*

$$\begin{aligned}
& \frac{}{\mathbf{0} \cdot \Gamma, x : [T]_1 \vdash x : T} \text{T-VAR} \\
& \frac{\Gamma, x : [T]_\ell \vdash t : T_2}{\Gamma \vdash \lambda x : T_1. t : T_1 \rightarrow^\ell T_2} \text{T-ABS} \\
& \frac{\Gamma_1 \vdash t_1 : T_{11} \rightarrow^\ell T_{12} \quad \Gamma_2 \vdash t_2 : T_{11}}{\Gamma_1 + \ell \cdot \Gamma_2 \vdash t_1 t_2 : T_{12}} \text{T-APP} \\
& \frac{\Gamma \vdash t : T}{\ell \cdot \Gamma \vdash [t] : \square_\ell T} \text{T-PRO} \\
& \frac{\Gamma_1 \vdash t_1 : \square_\ell T_1 \quad \Gamma_2, x : [T]_\ell \vdash t_2 : T_2}{\Gamma_1 + \Gamma_2 \vdash \mathbf{let} [x] = t_1 \mathbf{in} t_2 : T_2} \text{T-LET} \\
& \frac{\Gamma, x : [T]_{\ell_1} \vdash t : T_1 \quad \ell_2 \sqsubseteq \ell_1}{\Gamma, x : [T]_{\ell_2} \vdash t : T_1} \text{T-APPROX} \\
& \frac{\text{for each } i, \Gamma_i \vdash t_i : T_i}{\sum_{i=1}^n \Gamma_i \vdash \{l_i = t_i^{i \in 1..n}\} : \{l_i : T_i^{i \in 1..n}\}} \text{T-RECORD} \\
& \frac{\Gamma \vdash t : \{l_i : T_i^{i \in 1..n}\}}{\Gamma \vdash t.l_i : T_i} \text{T-PROJ} \\
& \frac{}{\mathbf{0} \cdot \Gamma \vdash \text{true} : \text{Bool}} \text{T-TRUE} \\
& \frac{}{\mathbf{0} \cdot \Gamma \vdash \text{false} : \text{Bool}} \text{T-FALSE} \\
& \frac{\Gamma_1 \vdash t_1 : \text{Bool} \quad \Gamma_2 \vdash t_2 : T \quad \Gamma_3 \vdash t_3 : T \quad \ell \sqsubseteq \mathbf{1}}{\ell \cdot \Gamma_1 + \Gamma_2 + \Gamma_3 \vdash \mathbf{if} t_1 \mathbf{then} t_2 \mathbf{else} t_3 : T} \text{T-COND} \\
& \frac{}{\mathbf{0} \cdot \Gamma \vdash 0 : \text{Nat}} \text{T-ZERO} \\
& \frac{\Gamma \vdash t : \text{Nat}}{\Gamma \vdash \text{succ } t : \text{Nat}} \text{T-SUCC} \\
& \frac{\Gamma \vdash t : \text{Nat}}{\Gamma \vdash \text{pred } t : \text{Nat}} \text{T-PRED} \\
& \frac{\Gamma \vdash t : \text{Nat}}{\Gamma \vdash \text{iszero } t : \text{Bool}} \text{T-ISZERO} \\
& \frac{}{\mathbf{0} \cdot \Gamma \vdash () : \text{Unit}} \text{T-UNIT} \\
& \frac{\Gamma_1 \vdash t_1 : \text{Unit} \quad \Gamma_2 \vdash t_2 : T}{\Gamma_1 + \Gamma_2 \vdash \mathbf{let} () = t_1 \mathbf{in} t_2 : T} \text{T-UNIT-ELIM}
\end{aligned}$$

5) *Evaluation Rules:*

$$\begin{aligned}
& \frac{t_1 \rightsquigarrow t'_1}{t_1 t_2 \rightsquigarrow t'_1 t_2} \text{E-APP1} \\
& \frac{t_2 \rightsquigarrow t'_2}{v_1 t_2 \rightsquigarrow v_1 t'_2} \text{E-APP2} \\
& \frac{}{(\lambda x : T. t) v \rightsquigarrow [x \mapsto v] t} \text{E-APPABS} \\
& \frac{t \rightsquigarrow t'}{[t] \rightsquigarrow [t']} \text{E-PRO} \\
& \frac{t_1 \rightsquigarrow t'_1}{\mathbf{let} [x] = t_1 \mathbf{in} t_2 \rightsquigarrow \mathbf{let} [x] = t'_1 \mathbf{in} t_2} \text{E-LET-EVAL} \\
& \frac{}{\mathbf{let} [x] = [v] \mathbf{in} t \rightsquigarrow [x \mapsto v] t} \text{E-LET-UNBOX} \\
& \frac{}{\{l_i = v_i^{i \in 1..n}\}. l_j \rightsquigarrow v_j} \text{E-PROJRECORD} \\
& \frac{t \rightsquigarrow t'}{t.l \rightsquigarrow t'.l} \text{E-PROJ} \\
& \frac{t_j \rightsquigarrow t'_j}{\{l_i = v_i^{i \in 1..j-1}, l_j = t_j, l_k = t_k^{k \in j+1..n}\} \rightsquigarrow \{l_i = v_i^{i \in 1..j-1}, l_j = t'_j, l_k = t_k^{k \in j+1..n}\}} \text{E-RECORD} \\
& \frac{t_1 \rightsquigarrow t'_1}{\mathbf{if} t_1 \mathbf{then} t_2 \mathbf{else} t_3 \rightsquigarrow \mathbf{if} t'_1 \mathbf{then} t_2 \mathbf{else} t_3} \text{E-IF-EVAL} \\
& \frac{}{\mathbf{if} \text{true} \mathbf{then} t_2 \mathbf{else} t_3 \rightsquigarrow t_2} \text{E-IF-TRUE} \\
& \frac{}{\mathbf{if} \text{false} \mathbf{then} t_2 \mathbf{else} t_3 \rightsquigarrow t_3} \text{E-IF-FALSE} \\
& \frac{t_1 \rightsquigarrow t'_1}{\mathbf{let} () = t_1 \mathbf{in} t_2 \rightsquigarrow \mathbf{let} () = t'_1 \mathbf{in} t_2} \text{E-UNIT-ELIM-EVAL} \\
& \frac{}{\mathbf{let} () = () \mathbf{in} t \rightsquigarrow t} \text{E-UNIT-ELIM} \\
& \frac{t \rightsquigarrow t'}{\text{succ } t \rightsquigarrow \text{succ } t'} \text{E-SUCC} \\
& \frac{n \text{ is a canonical numeral}}{\text{succ } n \rightsquigarrow n + 1} \text{E-SUCC-NAT} \\
& \frac{t \rightsquigarrow t'}{\text{pred } t \rightsquigarrow \text{pred } t'} \text{E-PRED} \\
& \frac{t \rightsquigarrow t'}{\text{iszero } t \rightsquigarrow \text{iszero } t'} \text{E-ISZERO} \\
& \frac{}{\text{pred } 0 \rightsquigarrow 0} \text{E-PRED-ZERO} \\
& \frac{n \text{ is a canonical numeral}}{\text{pred } n + 1 \rightsquigarrow n} \text{E-PRED-POSITIVE} \\
& \frac{}{\text{iszero } 0 \rightsquigarrow \text{true}} \text{E-ISZERO-ZERO} \\
& \frac{n \text{ is a canonical numeral}}{\text{iszero } n + 1 \rightsquigarrow \text{false}} \text{E-ISZERO-POSITIVE}
\end{aligned}$$

6) Bidirectional Typing:

$$\begin{array}{c}
\frac{\Phi \vdash t \Rightarrow T_1 \dashv \Delta \quad T_1 = T_2}{\Phi \vdash t \Leftarrow T_2 \dashv \Delta} \text{BD-SWITCH} \\
\\
\frac{\Phi(x) = T}{\Phi \vdash x \Rightarrow T \dashv \{x : \mathbf{1}\}} \text{BD-VAR} \\
\\
\frac{\Phi, x : T_1 \vdash t \Leftarrow T_2 \dashv \Delta \quad r \leq \Delta(x)}{\Phi \vdash \lambda x : T_1. t \Leftarrow T_1 \rightarrow^r T_2 \dashv \Delta \setminus \{x\}} \text{BD-ABS} \\
\\
\frac{\Phi \vdash t_1 \Rightarrow T_1 \rightarrow^r T_2 \dashv \Delta_1 \quad \Phi \vdash t_2 \Leftarrow T_1 \dashv \Delta_2}{\Phi \vdash t_1 t_2 \Rightarrow T_2 \dashv \Delta_1 + (r \cdot \Delta_2)} \text{BD-APP} \\
\\
\frac{\Phi \vdash t \Leftarrow T \dashv \Delta}{\Phi \vdash [t] \Leftarrow \Box_r T \dashv r \cdot \Delta} \text{BD-PRO} \\
\\
\frac{\Phi \vdash t_1 \Rightarrow \Box_r T \dashv \Delta_1 \quad \Phi, x : T \vdash t_2 \Leftarrow T_2 \dashv \Delta_2 \quad r \leq \Delta_2(x)}{\Phi \vdash \mathbf{let} [x] = t_1 \mathbf{in} t_2 \Leftarrow T_2 \dashv \Delta_1 + (\Delta_2 \setminus \{x\})} \text{BD-LET} \\
\\
\frac{\text{for each } i, \Phi \vdash t_i \Leftarrow T_i \dashv \Delta_i}{\Phi \vdash \{l_i = t_i^{i \in 1..n}\} \Leftarrow \{l_i : T_i^{i \in 1..n}\} \dashv \sum_{i=1}^n \Delta_i} \text{BD-RECORD} \\
\\
\frac{\Phi \vdash t \Rightarrow \{l_i : T_i^{i \in 1..n}\} \dashv \Delta}{\Phi \vdash t.l_i \Rightarrow T_i \dashv \Delta} \text{BD-PROJ} \\
\\
\frac{}{\Phi \vdash \mathbf{true} \Rightarrow \mathbf{Bool} \dashv \emptyset} \text{BD-TRUE} \\
\\
\frac{}{\Phi \vdash \mathbf{false} \Rightarrow \mathbf{Bool} \dashv \emptyset} \text{BD-FALSE} \\
\\
\frac{\Phi \vdash t \Leftarrow \mathbf{Bool} \dashv \Delta_1 \quad \Phi \vdash t_1 \Leftarrow T \dashv \Delta_2 \quad \Phi \vdash t_2 \Leftarrow T \dashv \Delta_3}{\Phi \vdash \mathbf{if } t \mathbf{ then } t_1 \mathbf{ else } t_2 \Leftarrow T \dashv \Delta_1 + \Delta_2 + \Delta_3} \text{BD-COND} \\
\\
\frac{}{\Phi \vdash 0 \Rightarrow \mathbf{Nat} \dashv \emptyset} \text{BD-ZERO} \\
\\
\frac{\Phi \vdash t \Leftarrow \mathbf{Nat} \dashv \Delta}{\Phi \vdash \mathbf{succ } t \Rightarrow \mathbf{Nat} \dashv \Delta} \text{BD-SUCC} \\
\\
\frac{\Phi \vdash t \Leftarrow \mathbf{Nat} \dashv \Delta}{\Phi \vdash \mathbf{pred } t \Rightarrow \mathbf{Nat} \dashv \Delta} \text{BD-PRED} \\
\\
\frac{\Phi \vdash t \Leftarrow \mathbf{Nat} \dashv \Delta}{\Phi \vdash \mathbf{iszero } t \Rightarrow \mathbf{Bool} \dashv \Delta} \text{BD-ISZERO} \\
\\
\frac{}{\Phi \vdash () \Rightarrow \mathbf{Unit} \dashv \emptyset} \text{BD-UNIT} \\
\\
\frac{\Phi \vdash t_1 \Leftarrow \mathbf{Unit} \dashv \Delta_1 \quad \Phi \vdash t_2 \Leftarrow T \dashv \Delta_2}{\Phi \vdash \mathbf{let } () = t_1 \mathbf{ in } t_2 \Leftarrow T \dashv \Delta_1 + \Delta_2} \text{BD-UNIT-ELIM}
\end{array}$$

B. Proofs

1) Security Level Semiring:

Proof.

- **Associativity of addition:** $(a + b) + c = a + (b + c)$
This is trivial since the semiring addition is meet, which is associative in a lattice.
- **Commutativity of addition:** $a + b = b + a$
This is also trivial since meet is commutative in a lattice.
- **Additive identity:** $a + \mathbf{0} = a$ for all a
Since $0 = \text{Sec}$ and Sec is the maximum element of $\mathbb{S}$, we have

$$\begin{aligned}
\text{Sec} \sqcap \text{Sec} &= \text{Sec} \\
\text{Pub} \sqcap \text{Sec} &= \text{Pub}
\end{aligned} \tag{9}$$

- **Associativity of multiplication:** $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
This is trivial by the associativity of join operation.
- **Multiplication distributes over addition:** $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$ and $(a + b) \cdot c = (a \cdot c) + (b \cdot c)$
The distributivity trivially holds since $\mathbb{S}$ is a two-point lattice.
- **Multiplicative identity:** $a \cdot \mathbf{1} = a$ for all a
Since $1 = \text{Pub}$ and $\text{Pub} \sqcap \text{Sec}$, we have

$$\begin{aligned}
\text{Pub} \sqcup \text{Pub} &= \text{Pub} \\
\text{Pub} \sqcup \text{Sec} &= \text{Sec}
\end{aligned} \tag{10}$$

- **Multiplication by 0 annihilates:** $\mathbf{0} \cdot a = a \cdot \mathbf{0} = \mathbf{0}$ for all a
Since $0 = \text{Sec}$ and $\text{Pub} \sqcap \text{Sec}$, we have

$$\begin{aligned}
\text{Sec} \sqcup \text{Sec} &= \text{Sec} \\
\text{Pub} \sqcup \text{Sec} &= \text{Sec}
\end{aligned} \tag{11}$$

Thus the security level algebra is a semiring. $\square$

2) Unary Fundamental Theorem:

Proof. By induction on the typing derivation of $\Gamma \vdash t : T$

- **Case T-VAR:** $\mathbf{0} \cdot \Gamma, x : [T]_1 \vdash x : T$.
Let $\gamma \in \llbracket \mathbf{0} \cdot \Gamma, x : [T]_1 \rrbracket$. The context relation gives $[\gamma(x)] \in \llbracket \Box_1 T \rrbracket_{\mathcal{E}}$. By the definition of the modality relation (and grade-erasure), this implies $\gamma(x) \in \llbracket T \rrbracket_{\mathcal{E}}$. Since the term is just x , $\gamma t = \gamma(x)$, satisfying the goal.
- **Case T-ABS:** $\Gamma \vdash \lambda x : T_1. t : T_1 \rightarrow^r T_2$ derived from $\Gamma, x : [T_1]_r \vdash t : T_2$.
Assume $\gamma \in \llbracket \Gamma \rrbracket$; we must show that $\lambda x : T_1. t \in \llbracket T_1 \rightarrow^r T_2 \rrbracket_{\mathcal{V}}$. Let v be a value with $[v] \in \llbracket \Box_r T_1 \rrbracket_{\mathcal{E}}$. Grade-erasure lets us extend γ with $x \mapsto v$ to a substitution in $\llbracket \Gamma, x : [T_1]_r \rrbracket$. By the induction hypothesis on the premise $\Gamma, x : [T_1]_r \vdash t : T_2$, we obtain $[x \mapsto v]t \in \llbracket T_2 \rrbracket_{\mathcal{E}}$, which is the required function-value condition.
- **Case T-APP:** $\Gamma_1 + r \cdot \Gamma_2 \vdash t_1 t_2 : T_2$.
Assume $\gamma \in \llbracket \Gamma_1 + r \cdot \Gamma_2 \rrbracket$. By the context-combination and grade-erasure lemmas, γ satisfies both premise contexts. The induction hypotheses place the function and argument in their respective expression relations. To verify the expression relation, consider any terminating

evaluation of the application: call-by-value evaluation then gives terminating evaluations of the function and argument to related values. The function-value clause and the application reduction rule give the result in $\llbracket T_2 \rrbracket_{\mathcal{E}}$; no termination of an arbitrary semantic substitution is assumed.

- *Case T-PRO:* $r \cdot \Gamma \vdash [t] : \Box_r T$.
By the induction hypothesis, $\gamma \ t \in \llbracket T \rrbracket_{\mathcal{E}}$. Evaluation compatibility gives $[\gamma \ t] \rightsquigarrow^* [v]$ whenever $\gamma \ t \rightsquigarrow^* v$ with $v \in \llbracket T \rrbracket_{\mathcal{V}}$; hence the boxed term is in $\llbracket \Box_r T \rrbracket_{\mathcal{E}}$.
- *Case T-LET:* $\Gamma_1 + \Gamma_2 \vdash \mathbf{let} \ [x] = t_1 \ \mathbf{in} \ t_2 : T_2$.
By the induction hypothesis, $\gamma \ t_1 \in \llbracket \Box_r T_1 \rrbracket_{\mathcal{E}}$. To verify the expression relation, consider any terminating evaluation of the let-term. Call-by-value evaluation then gives $\gamma \ t_1 \rightsquigarrow^* [v]$ with $[v]$ a canonical boxed value and $v \in \llbracket T_1 \rrbracket_{\mathcal{V}}$. Extend γ with $x \mapsto v$ and apply the induction hypothesis to t_2 ; the let-elimination step then preserves the expression relation. No termination of an arbitrary semantic substitution is assumed.
- *Case T-APPROX:* $\Gamma, x : [T_1]_s : t : T_2$ where $r \leq s$.
The unary relation for $\Box_r T$ does not depend on the value of r . Thus, if $\gamma(x) \in \llbracket \Box_s T_1 \rrbracket_{\mathcal{E}}$, it is also in $\llbracket \Box_r T_1 \rrbracket_{\mathcal{E}}$. The induction hypothesis applies directly.
- *Case T-RECORD:* $\sum_i \Gamma_i \vdash \{l_i = t_i^{i \in 1..n}\} : \{l_i : T_i^{i \in 1..n}\}$.
By context-combination compatibility, γ satisfies each field context Γ_i . The induction hypothesis therefore gives $\gamma \ t_i \in \llbracket T_i \rrbracket_{\mathcal{E}}$ for every i ; any terminating record has fields in the corresponding value relations, so it belongs to $\llbracket \{l_i : T_i^{i \in 1..n}\} \rrbracket_{\mathcal{E}}$.
- *Case T-PROJ:* $\Gamma \vdash t.l_i : T_i$.
By the induction hypothesis, every terminating evaluation of $\gamma \ t$ yields a record $\{l_i = v_i^{i \in 1..n}\}$ with $v_i \in \llbracket T_i \rrbracket_{\mathcal{V}}$. The projection rule then yields $\gamma \ (t.l_i) \in \llbracket T_i \rrbracket_{\mathcal{E}}$.
- *Case T-TRUE:* $\mathbf{0} \cdot \Gamma \vdash \mathbf{true} : \mathbf{Bool}$.
This follows immediately from the definition $\mathbf{true} \in \llbracket \mathbf{Bool} \rrbracket_{\mathcal{V}}$.
- *Case T-FALSE:* $\mathbf{0} \cdot \Gamma \vdash \mathbf{false} : \mathbf{Bool}$.
This follows immediately from the definition $\mathbf{false} \in \llbracket \mathbf{Bool} \rrbracket_{\mathcal{V}}$.
- *Case T-COND:* $r \cdot \Gamma_1 + \Gamma_2 + \Gamma_3 \vdash \mathbf{if} \ t_1 \ \mathbf{then} \ t_2 \ \mathbf{else} \ t_3 : T_1$.
By context-combination compatibility, γ satisfies each premise context. The induction hypothesis gives $\gamma \ t_1 \in \llbracket \mathbf{Bool} \rrbracket_{\mathcal{E}}$. If it evaluates to $\mathbf{true}$ (respectively, $\mathbf{false}$), the corresponding branch evaluates in $\llbracket T_1 \rrbracket_{\mathcal{E}}$ by the induction hypothesis, and the conditional evaluation preserves that relation.
- *Case T-ZERO:* $\mathbf{0} \cdot \Gamma \vdash 0 : \mathbf{Nat}$.
This follows immediately from the definition $0 \in \llbracket \mathbf{Nat} \rrbracket_{\mathcal{V}}$.
- *Case T-SUCC:* $\Gamma \vdash \mathbf{succ} \ t : \mathbf{Nat}$.
By the induction hypothesis and the successor rule, the result reduces to the canonical numeral $n + 1$, which is in $\llbracket \mathbf{Nat} \rrbracket_{\mathcal{V}}$.
- *Case T-PRED:* $\Gamma \vdash \mathbf{pred} \ t : \mathbf{Nat}$.
By the induction hypothesis and the predecessor rules,

the term reduces to a canonical numeral (either 0 or n), hence belongs to $\llbracket \mathbf{Nat} \rrbracket_{\mathcal{E}}$.

- *Case T-ISZERO:* $\Gamma \vdash \mathbf{iszero} \ t : \mathbf{Bool}$.
By the induction hypothesis and the zero-test rules, the term reduces to either $\mathbf{true}$ or $\mathbf{false}$, both of which are in $\llbracket \mathbf{Bool} \rrbracket_{\mathcal{V}}$.
- *Case T-UNIT:* $\mathbf{0} \cdot \Gamma \vdash () : \mathbf{Unit}$.
This follows immediately from the definition $() \in \llbracket \mathbf{Unit} \rrbracket_{\mathcal{V}}$.
- *Case T-UNIT-ELIM:* $\Gamma_1 + \Gamma_2 \vdash \mathbf{let} \ () = t_1 \ \mathbf{in} \ t_2 : T_2$.
For any terminating evaluation of the unit-elimination term, call-by-value evaluation first gives $\gamma \ t_1 \rightsquigarrow^* ()$. The induction hypothesis on t_2 then yields the result in $\llbracket T_2 \rrbracket_{\mathcal{E}}$. $\square$

3) Binary Fundamental Theorem:

Proof. By induction on the typing derivation.

- *Case T-VAR:* $\mathbf{0} \cdot \Gamma, x : [T]_1 \vdash x : T$.
The context relation gives the inputs in $\llbracket \Box_1 T \rrbracket_{\mathcal{E}}^{adv}$. Since $1 \leq adv$ by minimality of 1 , this is the observable modality case, which yields related underlying values at T .
- *Case T-ABS:* $\Gamma \vdash \lambda x : T_1. t : T_1 \rightarrow^r T_2, \Gamma, x : [T_1]_r \vdash t : T_2$.
For $(u_1, u_2) \in \llbracket \Box_r T_1 \rrbracket_{\mathcal{E}}^{adv}$, extend the two related substitutions with the underlying values and apply the induction hypothesis to obtain $([x \mapsto u_1]t, [x \mapsto u_2]t) \in \llbracket T_2 \rrbracket_{\mathcal{E}}^{adv}$, which is the function-value condition.
- *Case T-APP:* $\Gamma_1 + r \cdot \Gamma_2 \vdash t_1 \ t_2 : T_2$.
By the induction hypotheses and the context-combination/scaling lemmas, the function and argument pairs are in their respective expression relations. Reduction compatibility lifts the argument relation to the required pair of boxed values, and the binary function-value clause yields the result in $\llbracket T_2 \rrbracket_{\mathcal{E}}^{adv}$.
- *Case T-PRO:* $r \cdot \Gamma \vdash [t] : \Box_r T$.
We have 2 subcases to consider:
 - *Subcase $r \leq adv$:* The required context-relatedness follows from the explicit monotonicity and compatibility lemmas for context scaling; these lemmas must be proved for the chosen semiring (they do not follow from weak idempotence alone). Applying the induction hypothesis then gives the related promoted values.
 - *Subcase $\neg(r \leq adv)$:* both promoted results satisfy the unary relation by the Unary Fundamental Theorem, which is exactly the unobservable branch of the modality relation.
- *Case T-LET:* $\Gamma_1 + \Gamma_2 \vdash \mathbf{let} \ [x] = t_1 \ \mathbf{in} \ t_2 : T_2$.
If the eliminated modality has grade $r \leq adv$, the unwrapped values are related by the observable case of the modality binary relation, and the induction hypothesis applies to the body. If $\neg(r \leq adv)$, the two unwrapped values need only satisfy the unary relation;

the body case therefore requires the corresponding non-observability/compatibility lemma for the graded context, rather than any claim about the result type's grade.

- *Case T-APPROX*: $\Gamma, x : [T_1]_s : t : T_2$ where $r \leq s$.
The conclusion context has the lower grade r , with $r \leq s$. Related substitutions at grade r also satisfy the premise context at grade s , because the binary modality relation becomes weaker as the grade increases. The induction hypothesis therefore applies to the premise derivation.
- *Case T-RECORD*: $\sum_i \Gamma_i \vdash \{l_i = t_i^{i \in 1..n}\} : \{l_i : T_i^{i \in 1..n}\}$.
By context-combination compatibility, both related substitutions satisfy each field context Γ_i . The induction hypotheses give related field evaluations, and combining them yields $(\{l_i = t_i^{i \in 1..n}\}, \{l_i = t_i^{i \in 1..n}\}) \in \llbracket \{l_i : T_i^{i \in 1..n}\} \rrbracket_{\mathcal{E}}^{adv}$.
- *Case T-PROJ*: $\Gamma \vdash t.l_i : T_i$.
By the induction hypothesis, both scrutinees reduce to related record values. Reduction compatibility and the projection closure lemma then imply that their projections are related at T_i ; no syntactic equality of the two records is required.
- *Case T-TRUE*: $\mathbf{0} \cdot \Gamma \vdash \text{true} : \text{Bool}$.
This follows immediately from the definition $(\text{true}, \text{true}) \in \llbracket \text{Bool} \rrbracket_{\mathcal{V}}^{adv}$.
- *Case T-FALSE*: $\mathbf{0} \cdot \Gamma \vdash \text{false} : \text{Bool}$.
This follows immediately from the definition $(\text{false}, \text{false}) \in \llbracket \text{Bool} \rrbracket_{\mathcal{V}}^{adv}$.
- *Case T-COND*: $r \cdot \Gamma_1 + \Gamma_2 + \Gamma_3 \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T_1$.

We have two subcases to consider:

- *Subcase $r \leq adv$* : Then $\gamma_1 t_1$ and $\gamma_2 t_1$ must result in the same boolean value. Thus both execution take the same branch, and the induction hypothesis on the executed branch applies.
- *Subcase $\neg(r \leq adv)$* : The branches may differ. The result is related only via the unobservable case, using the unary fundamental theorem and the compatibility lemma for conditionals; no result grade is implied by T-COND.

- *Case T-ZERO*: $\mathbf{0} \cdot \Gamma \vdash 0 : \text{Nat}$.
This follows immediately from the definition $(0, 0) \in \llbracket \text{Nat} \rrbracket_{\mathcal{V}}^{adv}$.
- *Case T-SUCC*: $\Gamma \vdash \text{succ } t : \text{Nat}$.
The two evaluations reduce, by compatibility, to canonical numerals; the numeral relation relates identical canonical forms.
- *Case T-PRED*: $\Gamma \vdash \text{pred } t : \text{Nat}$.
The predecessor evaluations reduce to identical canonical numerals, which are related at Nat .
- *Case T-ISZERO*: $\Gamma \vdash \text{iszero } t : \text{Bool}$.
The two zero tests reduce to identical booleans, which are related at Bool .
- *Case T-UNIT*: $\mathbf{0} \cdot \Gamma \vdash () : \text{Unit}$.
This follows immediately from the definition $((), ()) \in \llbracket \text{Unit} \rrbracket_{\mathcal{V}}^{adv}$.

- *Case T-UNIT-ELIM*: $\Gamma_1 + \Gamma_2 \vdash \text{let } () = t_1 \text{ in } t_2 : T_2$.
Follows immediately from the induction hypothesis on the body t_2 . □

4) Noninterference of FLOWSTLC:

Proof. Because $adv \leq r$ and $adv \neq r$, antisymmetry gives $\neg(r \leq adv)$. The unary fundamental theorem places each closed input value in $\llbracket T_1 \rrbracket_{\mathcal{E}}$, so the pair of substitutions is related at the unobservable modality $\Box_r T_1$. Applying the binary fundamental theorem yields $([x \mapsto v_1]t, [x \mapsto v_2]t) \in \llbracket \Box_{adv} \mathbb{B} \rrbracket_{\mathcal{E}}^{adv}$. Since $adv \leq adv$, any terminating results are related at the underlying base type. The base relation relates identical canonical observations, hence if the two evaluations terminate with $[b_1]$ and $[b_2]$, then $b_1 = b_2$. No equality of the complete terms is implied without an additional normalization/confluence argument. □